University of Science and Technology at Zewail City
Communications and Information Engineering

Cloud Computing – Lab 4 Assignment

Student Course Registration System

Multi-Tier Deployment with Docker Compose

Team member	Student ID
Abdelhady Mohamed	202201172
Yousef Allam	202200286
Abdulrahman Magdy	202200341

Assigned host port	8072 (8000 + 72)
Repository	https://github.com/3bdelhadYy/Multi-Tier-Application-with-Docker-Compose
Environment	Ubuntu on WSL2 with Docker Compose

October 2026

Contents

1 Introduction and Objectives **2**

2 System Architecture **2**

3 Deployment and Main Commands **2**

4 Networking and Port Configuration **3**

5 Database and Persistence **3**

6 Testing, Troubleshooting, and Recovery **3**

7 Screenshots and Results **3**
7.1 Environment and Deployment 4
7.2 Application and Database 4
7.3 Network and Health Check 5
7.4 Persistence and Recovery 5

8 Conclusion **6**

1 Introduction and Objectives

This lab deploys a Student Course Registration System using Docker Compose. The application lets users view courses, manage students, and register students in courses. The main goals are to run a three-tier application, separate services with Docker networks, publish only the assigned frontend port, preserve database records using a named volume, and demonstrate basic monitoring and recovery.

2 System Architecture

The system contains three services:

- **Frontend:** Nginx serves the HTML, CSS, and JavaScript interface and forwards `/api/` requests to the backend.
- **Backend:** FastAPI handles application requests and communicates with PostgreSQL.
- **Database:** PostgreSQL stores course, student, and registration records.

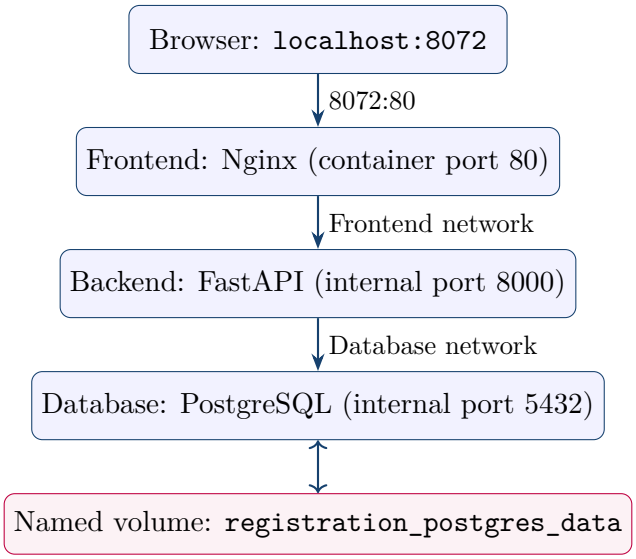


Figure 1: Three-tier application architecture and persistent database storage.

The frontend and backend share `registration_frontend_network`. The backend and database share `registration_database_network`. The frontend is not attached to the database network. Only the frontend publishes a host port; the backend and database are intended for internal communication.

3 Deployment and Main Commands

The services are defined in `compose.yaml`. Compose builds the frontend and backend images, starts PostgreSQL, checks database readiness, and connects the services to their required networks. The database uses a named volume so that its data is not stored only in the container's writable layer.

The following are the main commands used during setup and deployment:

```
docker --version
docker compose version
docker run --rm hello-world
docker images
docker compose up -d --build
docker compose ps
```

Open the application at `http://localhost:8072`. The expected services are frontend, backend, and database. The database should become healthy before the backend relies on it.

4 Networking and Port Configuration

The project uses two bridge networks:

- `registration_frontend_network`: frontend and backend.
- `registration_database_network`: backend and database.

This segmentation limits direct connectivity: the frontend does not join the database network. The frontend is published as host port 8072 to container port 80. Backend port 8000 and PostgreSQL port 5432 are not published to the host.

```
docker network inspect registration_frontend_network
docker network inspect registration_database_network
curl -i http://localhost:8072/api/health
```

5 Database and Persistence

The database stores records in the `students`, `courses`, and `registrations` tables. The application uses relational constraints to protect data integrity and prevents duplicate student-course registrations. Course capacity is checked by the backend when registrations are submitted.

The PostgreSQL data directory is backed by the named volume `registration_postgres_data`. To test persistence, the application data was queried before and after removing and recreating containers. The important safety note is to use `docker compose down`, not `docker compose down -v`, because `-v` removes named volumes.

```
docker exec -it registration-db psql -U postgres_user -d course_registration_db -c "\dt"
docker compose down
docker compose up -d
```

6 Testing, Troubleshooting, and Recovery

The health endpoint `/api/health` is used to check application and database availability. To demonstrate troubleshooting, the database service can be stopped, backend logs inspected, and the database restarted. The exact response should be described according to the captured output.

```
docker compose stop db
curl -i http://localhost:8072/api/health
docker compose logs --tail=50 backend
docker compose start db
docker compose ps
```

After restarting the database, check that it is healthy and retry the health endpoint. Logs help identify database connectivity problems. Do not remove the volume during this test.

7 Screenshots and Results

The following screenshots document the setup, application functionality, database, networks, persistence test, and troubleshooting. Captions describe the evidence; the images themselves should be checked against the actual observed outputs.

7.1 Environment and Deployment

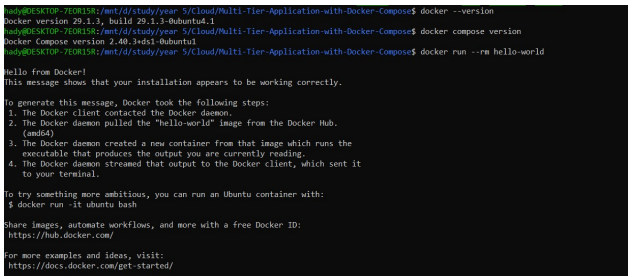


Figure 2: Docker and Compose versions and the hello-world test.

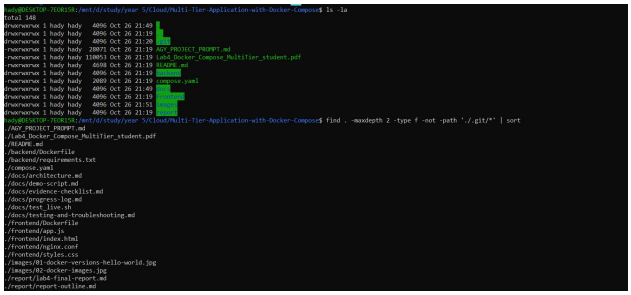


Figure 4: Project directory structure.

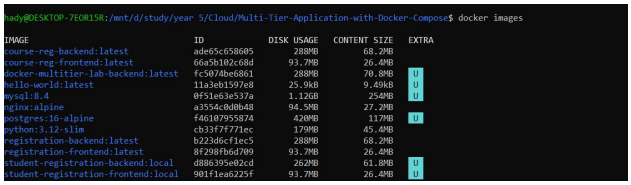


Figure 3: Available Docker images.

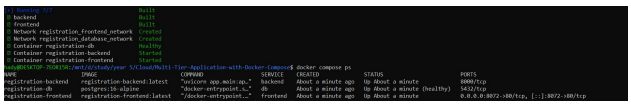


Figure 5: Compose service status.

7.2 Application and Database

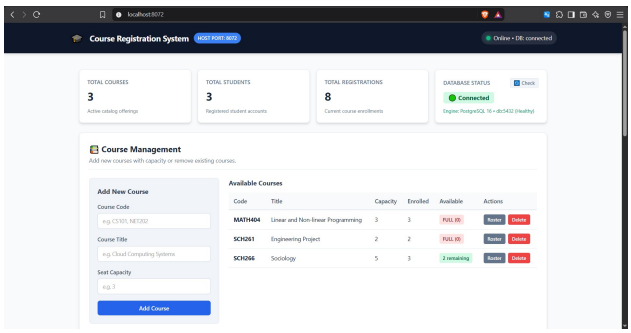


Figure 6: Course management interface.

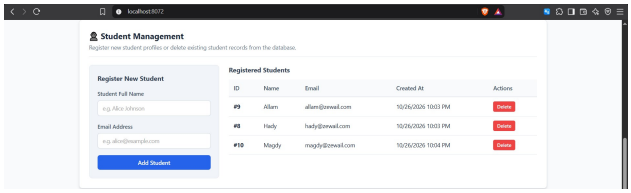


Figure 7: Student management interface.

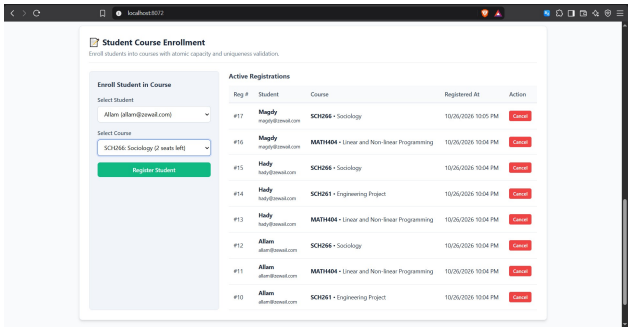


Figure 8: Registration records and course roster.

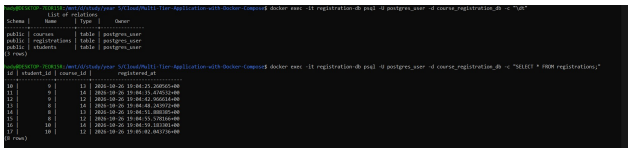


Figure 9: PostgreSQL table and registration inspection.

7.3 Network and Health Check

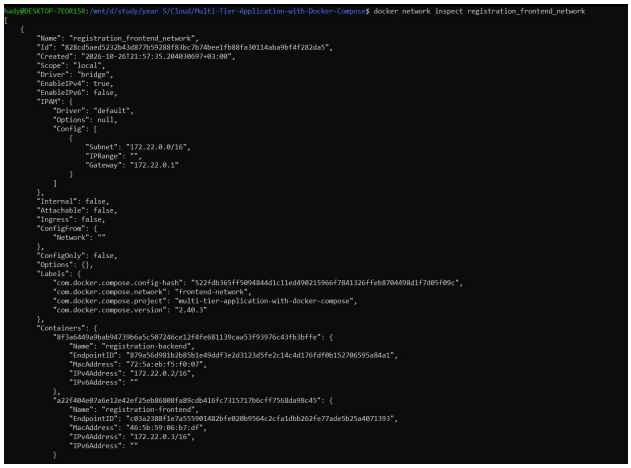


Figure 10: Frontend network inspection.

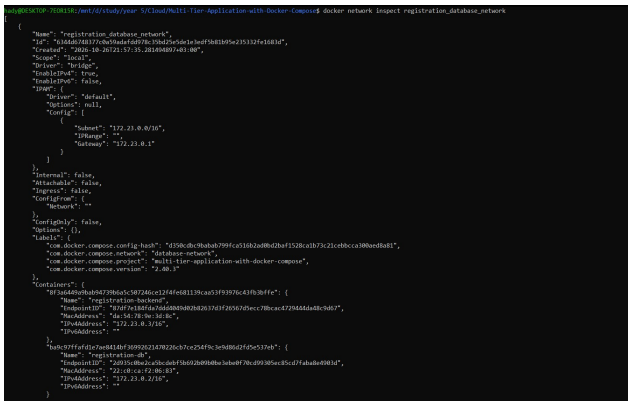


Figure 11: Database network inspection.

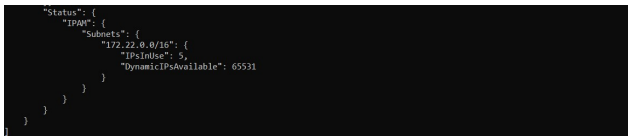


Figure 12: Additional frontend network evidence.



Figure 13: Additional database network evidence.

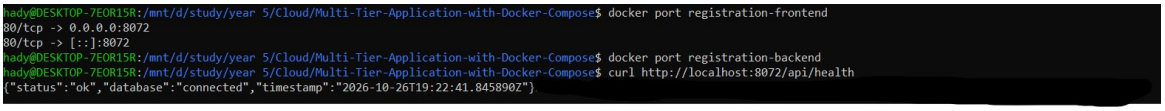


Figure 14: Port mapping and API health check.

7.4 Persistence and Recovery



Figure 15: Data before container recreation.

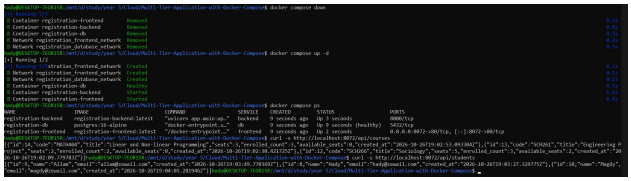


Figure 16: Data after container recreation.

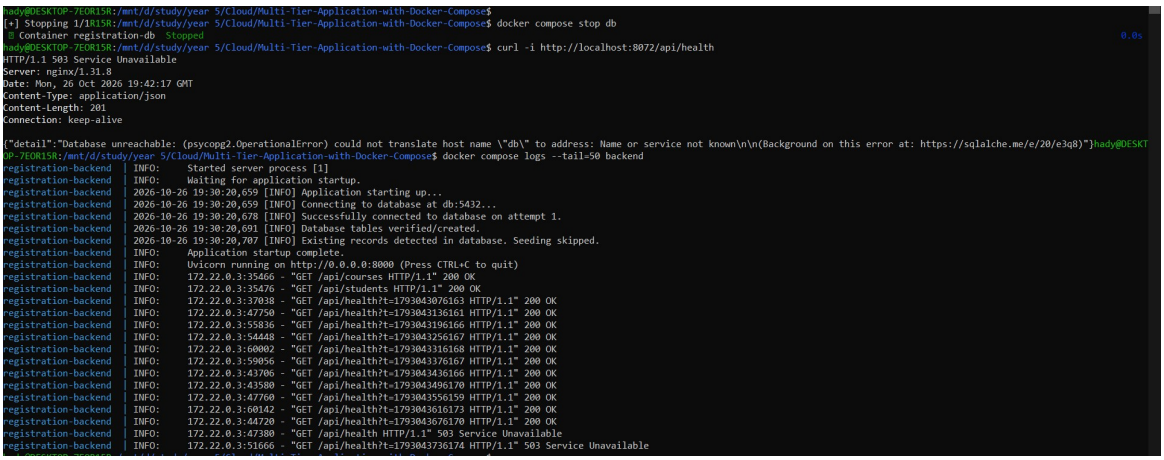


Figure 17: Database failure, logs, and recovery evidence.

8 Conclusion

The project demonstrates a three-tier application managed by Docker Compose. Nginx provides the single host entry point on port 8072, FastAPI handles application logic, and PostgreSQL stores relational data. Separate bridge networks limit direct connectivity between tiers, while a named volume supports data persistence across container recreation. The screenshots and commands provide evidence of deployment, application use, database inspection, and troubleshooting.